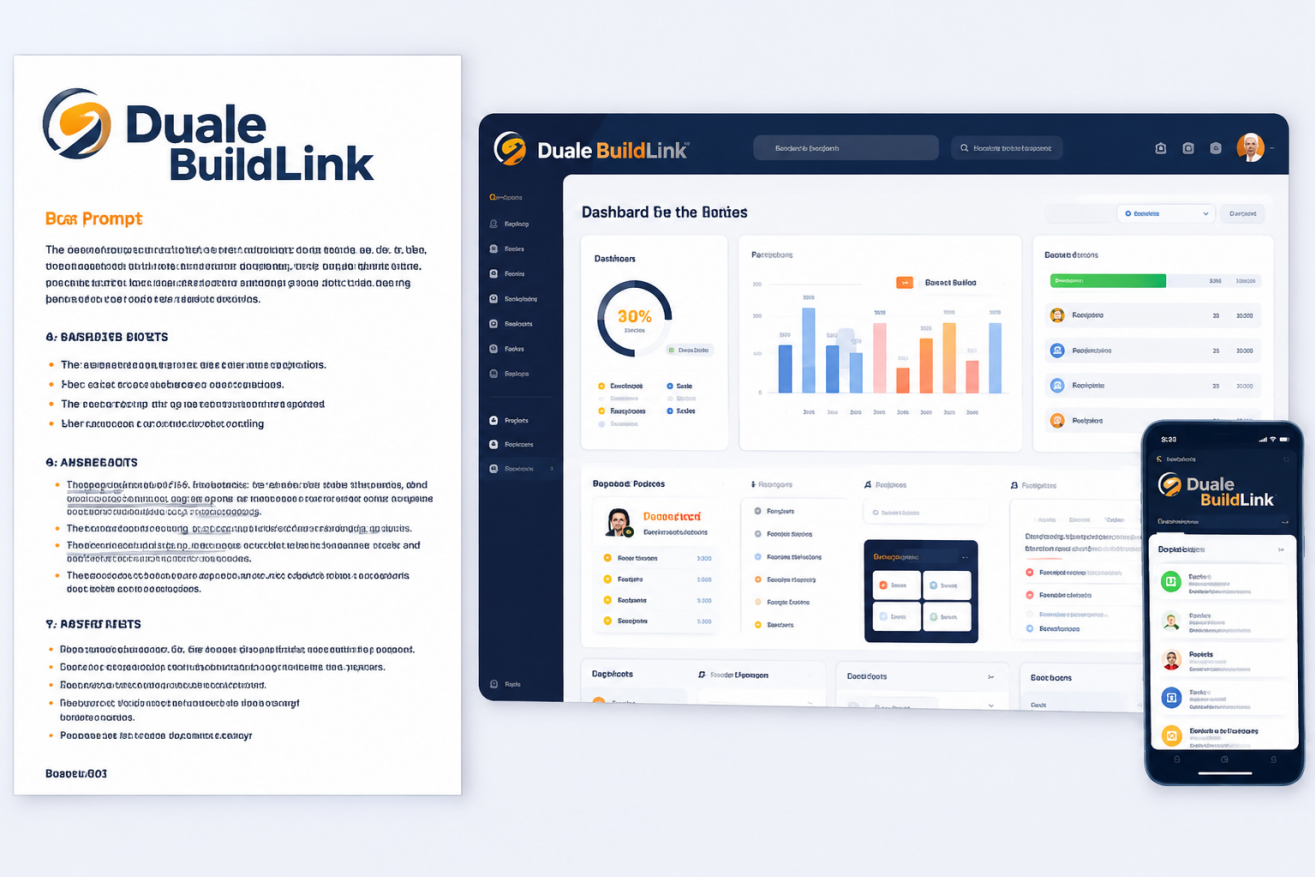
DUALE BUILDLINK

Production-Grade Technician Management & Job-Dispatch Platform

Complete implementation prompt • Functional requirements • Architecture • Database • Security • Testing • Deployment



Product vision

Duale BuildLink connects furniture retailers, plumbing and electrical suppliers, building-material companies, construction businesses, and other service providers with capable technicians. Companies create customer jobs, select or receive recommendations for eligible technicians, track service delivery, and reconcile job-related finances.

Brand direction Navy blue, white, and construction orange; clear operational dashboards; responsive company portal; mobile-first technician experience.

Status: Proposed specification. Local policies, payment providers, legal requirements, hosting, and business rules must be verified before production launch.

1. Master Implementation Prompt

Build **Duale BuildLink**, a secure, production-grade, multi-company technician management and job-dispatch platform. It must be a working operational system, not a static dashboard, demo, disconnected CRUD application, or prototype that merely looks complete.

Deliver the complete source code, database schema and migrations, API, company web dashboard, technician mobile experience, platform administration interface, automated tests, deployment configuration, seed/reference data, and documentation. Implement end-to-end workflows and verify them with tests.

Core operating model

• **Hybrid technician network:** companies can manage their own technicians and use approved independent technicians who may work with multiple companies under explicit company-specific relationships.

• **Company-confirmed dispatch:** the system recommends eligible technicians based on required skills, verification, relationship, availability, service area, workload, and configurable ranking factors. Authorized company staff confirm assignments. Recommendations must not silently become assignments.

• **Flexible payment model:** support company-paid technicians and direct customer-to-technician payment arrangements. Clearly record who owes whom, the agreed amount, payment status, evidence, and reconciliation state.

• **Customer role:** customers are service recipients and do not need accounts in the first release. Design for an optional future customer portal.

2. Users, Roles, and Access Control

• **Platform administrator:** platform configuration, approved business categories, service areas, account status, and controlled operational oversight.

• **Verification officer:** review company and technician identity or qualification evidence; approve, reject, or request more information.

• **Platform finance officer:** platform subscriptions, platform fees, refunds, and reconciliation within explicitly granted scope.

• **Company owner/administrator:** company profile, branches, staff, technician relationships, and settings.

• **Dispatcher:** create jobs, view eligible candidates, confirm assignments, reschedule, cancel, and reassign.

• **Service supervisor:** review evidence, approve completion, return work for correction, and manage callbacks.

• **Company finance officer:** review invoices, payments, technician payables, refunds, and reconciliation.

• **Read-only auditor:** read authorized records without mutation.

• **Technician:** manage allowed profile data, availability, assigned jobs, status updates, evidence, own earnings, and disputes.

Enforce authorization on the backend for every request. Check identity, active status, permission, tenant/branch scope, technician-company relationship, resource state, and approval requirements. Hiding UI controls is not authorization. UUIDs are not access control.

3. Required Functional Modules

• **Company and branch management:** applications, verification, company profile, branches, membership, roles, service categories, service areas, suspension and reactivation.

• **Technician management:** onboarding, identity/qualification documents, trades, skills, verification status and expiry, company relationships, availability, service areas, history, and restrictions.

• **Trade and skill catalog:** configurable categories for furniture assembly, plumbing, electrical work, masonry, painting, tiling, carpentry, fittings, maintenance, and future trades.

• **Customer and location management:** company-scoped customer records, contact preferences, service addresses, optional coordinates, access instructions, and prior jobs.

• **Job management:** unique reference, company/branch, customer/location, required skills, scope, schedule, priority, charges, compensation terms, evidence, status history, complaints, and callbacks.

• **Matching and dispatch:** eligibility filtering, explainable candidate ranking, assignment confirmation, technician response, conflict checks, reassignment history, and notification.

• **Job execution:** accepted, en route, arrived, in progress, awaiting review, completed, returned for correction, cancelled, and disputed states; validate permitted transitions.

• **Quality and warranty:** completion evidence, supervisor review, complaints, linked callback jobs, warranty terms, rework responsibility, and resolution history.

• **Notifications:** in-app notifications and a persistent delivery queue; provider integrations only after selection and verification.

• **Reporting:** job volume/status, technician utilization, cancellation, acceptance, duration, complaints, collections, payables, outstanding balances, verification queues, and platform fees.

4. Workflow Requirements

• **Company onboarding:** register → submit required evidence → review → approve/reject/request information → configure staff and operations.

• **Technician onboarding:** register → profile/skills → submit evidence → verify → establish company relationship and terms → become eligible for matching.

• **Dispatch:** create and validate job → find eligible candidates → dispatcher selects/reviews recommendations → company confirms → technician is notified → acceptance/decline → confirm or reassign.

• **Execution:** technician receives minimum necessary customer information → records arrival/start → reports scope changes → submits work report and evidence → supervisor reviews.

• **Completion:** supervisor approves or returns work → completion is recorded → applicable financial workflow proceeds separately → feedback/callback eligibility is recorded.

• **Company-paid settlement:** approved compensation → payable → authorized review → payment initiation → trusted confirmation/evidence → reconciliation → settled.

• **Customer-paid settlement:** record customer as payer and technician as payee → communicate agreed charge → record cash acknowledgment or trusted provider confirmation → reconcile → handle disputes separately.

• **Cancellation/reassignment:** validate state and authority, record actor/reason/time, preserve previous assignment, apply approved charges or adjustments, and notify affected parties.

• **Disputes:** link job/payment, collect evidence, assign reviewer, record decision, post any approved adjustment separately, notify parties, preserve history.

5. Architecture and Technology

Use a modular monolith initially, with clear module boundaries and API contracts. Do not introduce microservices unless measured scale or operational needs justify them.

• **Company web dashboard:** React and TypeScript (proposed).

• **Technician app:** Flutter, Android-first (proposed); responsive mobile web can be used for early validation if appropriate.

• **Backend:** ASP.NET Core Web API (proposed).

• **Database:** PostgreSQL with Entity Framework Core (proposed).

• **Background processing:** persistent queue, retry handling, and idempotent workers.

• **Files:** private object storage with short-lived authorized links.

• **Authentication:** supported OpenID Connect/OAuth 2.0 identity provider or a securely implemented first-party system.

• **Operations:** Docker-based deployment, HTTPS reverse proxy, secrets management, centralized logs, metrics, error tracking, backups, and health checks.

• Keep business rules in the backend; validate all state transitions; use database transactions for assignments and financial operations; use an outbox pattern for reliable event delivery; isolate external integrations behind interfaces; store time consistently and money using decimal-safe types.

6. Database Design

Create normalized tables with primary keys, foreign keys, indexes, uniqueness constraints, audit history, and tenant-aware access. The following is the minimum logical schema; implementation must add concrete types, constraints, migrations, and indexes.

Identity and access

users; companies; company_memberships; roles; permissions; role_permissions; membership_roles; branches; branch_memberships.

Technicians and skills

technicians; technician_company_relationships; trades; skills; technician_skills; technician_documents; service_areas; technician_service_areas; technician_availability.

Customers

customers; service_locations.

Jobs and dispatch

jobs; job_required_skills; job_assignments; job_status_history; job_evidence; job_work_reports; job_reviews; job_complaints; job_relationships.

Finance

job_financial_terms; invoices; invoice_lines; payment_records; financial_allocations; technician_payables; refunds; financial_ledger_entries; platform_fee_records; disputes.

Platform and operations

notifications; audit_events; file_metadata; idempotency_records; integration_accounts; outbox_events; platform_subscriptions.

Key field requirements

• **users:** ID, email/phone where used, password hash or external identity reference, display name, account status, verification timestamps, audit timestamps.

• **companies:** ID, display name, business category, optional registration reference, contact details, verification and account status.

• **technician_company_relationships:** technician, company, relationship type/status, validity period, compensation-terms reference, audit timestamps.

• **technicians/skills/documents:** profile, skill verification and expiry, secure file reference, reviewer, review status, and relevant dates.

• **jobs:** unique reference, company/branch, customer/location, trade, description, priority, status, requested/scheduled times, creator, completion timestamp.

• **job_assignments:** job, technician, eligible relationship, assignment status, assigned by, offer/response/acceptance times, end time, reassignment reason. Preserve all previous assignments.

• **job_financial_terms:** job, version, customer charge, technician compensation, currency, payer/payee types, platform fee if applicable, approver and approval time.

• **payment_records:** payment reference, parties, method, provider reference, amount/currency, status, initiation and confirmation timestamps.

• **financial ledger:** immutable balanced entries by currency, account code, source reference, timestamp, and actor. Corrections use reversal/adjustment entries.

• **audit_events:** actor, action, resource type/ID, company scope, timestamp, correlation ID, and redacted metadata.

Database constraints must prevent orphaned records, unauthorized relationships, incompatible currency operations, duplicate payment posting, and invalid state transitions. Financial history and audit history must not be cascade-deleted. Prevent overlapping confirmed assignments transactionally.

7. Financial Controls

• Keep customer charges, customer collections, technician compensation, accrued payables, paid amounts, company revenue, direct customer-to-technician payments, platform fees, refunds, and balances distinct.

• Store currency on each financial transaction. Use NUMERIC(19,4) or integer minor units; never use floating-point arithmetic for money.

• Separate payment initiation from confirmation. Verify provider callbacks; use idempotency; keep attempt history; reconcile provider reports; record partial payments, refunds, reversals, and adjustments separately.

• Cash receipts should record the amount, date, acknowledging party, and responsible user; do not represent a self-reported cash record as independently verified transfer.

• Support configurable approval thresholds and separation of duties for sensitive refunds and financial adjustments.

• Reports must be traceable to source records and distinguish pending, failed, confirmed, reversed, refunded, and disputed transactions.

8. Security and Privacy

• Use secure password hashing, verified contact channels where required, session revocation, safe account recovery, rate limiting, and risk-appropriate MFA.

• Enforce tenant and branch authorization on every API endpoint and object/file access. Test cross-company identifier substitution and role escalation.

• Require HTTPS; protect secrets outside source control; restrict identity documents; use short-lived file links; validate and scan uploads; redact logs.

• Audit permission changes, verification decisions, dispatch, cancellation, completion, price/scope changes, payment actions, refunds, and administrative interventions.

• Address relevant OWASP risks including broken access control, injection, authentication weaknesses, insecure file handling, misconfiguration, and vulnerable dependencies.

• Confirm applicable legal, privacy, tax, business, employment, and payment-service requirements before launch. Do not claim compliance before verification.

9. Nonfunctional Requirements

• Initial proposed availability target: 99.5% monthly after stabilization.

• Proposed p95 target: below 500 ms for ordinary internal API operations, excluding external-provider latency and heavy reports.

• Support horizontal scaling of API instances and workers; monitor queue depth, failures, latency, database capacity, and storage.

• Proposed recovery point objective: no more than 15 minutes where infrastructure supports point-in-time recovery; proposed recovery time objective: within four hours. Validate through restore exercises.

• Persist background jobs and retry safely. External provider outages must not corrupt job or payment state.

• Provide accessibility-conscious responsive layouts, understandable errors, loading/empty states, and robust handling of unreliable mobile connectivity.

10. Testing and Acceptance

• Unit tests: permissions, tenant scoping, state transitions, matching, scheduling, calculations, payment allocation, verification expiry, and retry rules.

• Integration tests: migrations, database constraints, concurrent assignment, file access, provider callbacks, idempotency, notifications, and authentication.

• End-to-end tests: company onboarding through job dispatch, execution, review, payment, reconciliation, and reporting for both payment models.

• Security tests: cross-tenant access, role escalation, IDOR, injection, malicious uploads, rate limits, secret leakage, audit tampering, and duplicate payment callbacks.

• Resilience tests: provider outage, queue backlog, network interruption, database exhaustion, backup restoration, and mobile offline reconciliation.

• User acceptance tests with representative companies and technicians. A running app or polished dashboard alone is not evidence of production readiness.

11. Deployment and Release

• Maintain separate development, staging, and production environments; never use production secrets in source control.

• CI/CD: code checks → automated tests → dependency/security scans → build → migration validation → staging deployment → smoke/UAT checks → approved production deployment → health checks.

• Use safe, backward-compatible database migrations where possible; document rollback and forward-fix procedures.

• Production infrastructure requires HTTPS, application hosting, database, persistent queue, private file storage, secrets management, monitoring, backups, DNS/certificates, and restricted deployment access.

• Document backup restore, incident response, permission administration, verification, payment reconciliation, failed notifications, disputes, data requests, and release rollback.

• Perform a clean-install test using only documented deployment steps.

12. Implementation Phases

• **Phase 1 — Foundation:** authentication, company onboarding, RBAC, technician profiles, trades/skills, and tenant isolation.

• **Phase 2 — Relationships and dispatch:** hybrid relationships, availability, customers, jobs, matching, assignment, acceptance, and reassignment.

• **Phase 3 — Execution and quality:** technician mobile workflow, evidence, supervisor review, complaints, callbacks, and warranty records.

• **Phase 4 — Finance:** financial terms, invoices, payables, payment records, direct payments, platform fees, refunds, and reconciliation.

• **Phase 5 — Reporting and notifications:** operational/finance reports, reminders, retryable notifications, and platform oversight.

• **Phase 6 — Hardening:** security, performance, backup restoration, monitoring, clean installation, documentation, and pilot acceptance.

13. Production Release Gate

• Company and technician onboarding pass end-to-end tests.

• Both technician relationship models and company-confirmed recommendations work correctly.

• Concurrent dispatch cannot create conflicting confirmed assignments.

• Job status transitions and assignment histories are enforced.

• Both financial workflows reconcile correctly; duplicate payments are prevented.

• Tenant isolation, document protection, audit trails, and permission boundaries are tested.

• Backup restoration, monitoring, security scans, mobile layouts, and unstable-connectivity scenarios are tested.

• Deployment/recovery documentation is complete and pilot users approve the workflow.

• All release-blocking defects are resolved and unresolved legal/provider decisions are documented.

14. Business Decisions That Must Not Be Invented

• Initial launch city and service radius.

• Company eligibility and verification evidence.

• Trade-specific technician qualifications.

• Company/technician relationship terms and suspension rules.

• Compensation model: fixed, hourly, percentage, negotiated, or configurable.

• Platform revenue model: subscription, per-job fee, or another approved model.

• Supported cash, mobile-money, bank, or payment-provider workflows.

• Customer communication and consent policy.

• Cancellation, warranty, callback, and dispute policies.

• Hosting provider, domain, operational owner, retention periods, and approved technology stack.

15. Required Deliverables

• Complete source code with clear module boundaries.

• Company web dashboard, technician mobile experience, and platform admin interface.

• Backend API and documented endpoint contracts.

• Database migrations, seed/reference data, and schema documentation.

• Role/permission matrix and tenant-isolation tests.

• Automated unit, integration, security-oriented, and end-to-end tests.

• Environment configuration examples without secrets.

• Deployment, backup/restore, monitoring, and incident-response documentation.

• Known limitations, unresolved decisions, and production readiness report.

Final instruction to the implementation team

Build Duale BuildLink as a secure, auditable operational platform connecting a business sale or service request to an eligible technician, a completed job, and a traceable financial outcome. Prioritize correctness, company isolation, workflow integrity, and financial controls over decorative features. Clearly distinguish implemented functionality from mockups, assumptions, and integrations that still require configuration.